Using Python

Installing python

Download from the website and run through the entire install process run through using an administrator account with admin permissions.

Check the python version

```
C:\Users\zacbh>python --version
Python 3.14.3
```

Check python works in cmd

Hello world

```
C:\Users\zacbh>python
Python 3.14.3 (tags/v3.14.3:323c59a, Feb  3 2026, 16:04:56) [MSC v.1944 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> print("Hello World!")
Hello World!
>>> |
```

Using `python` will open the editor, then test you're in the editor by using a simple call like `print()` to confirm it's working.

The Zen of Python

```
>>> import this
The Zen of Python, by Tim Peters

Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one-- and preferably only one --obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than *right* now.
If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.
Namespaces are one honking great idea -- let's do more of those!
>>> |
```

Exit

```
>>> exit()

C:\Users\zacbh>|
```

Using `exit()` will exit the editor

Fundamentals

Variables

A variable is a mutable named storage location.

```
>>> variableNamedStorage = "value to store"
```

Constants

A constant is an immutable named storage location. Enforcing constants is not “pythonic”, as such, Python does not allow for constants.

Data types

Character

A character or a ‘char’ is a single unicode or ascii symbol, depending on the system.

String

A string is a ‘string of text’, as such any character can be included in a string.

Integer

An integer is a numerical whole number, i.e., 0, 1, 2 or 10, but not 1.2 as that is not a whole number. Integers can be negative or positive values.

Float

A float is a numeric value that includes fractional parts and decimal points, i.e., 1.2, 10.9 or even 1.0, but not 1. All floating point numbers must have a ‘floating’ or ‘decimal’ point, so 1 becomes 1.0.

Boolean

A boolean value is either True or False. Booleans are logical values that can have logical operations performed on them. Boolean values are base 2, meaning there will only ever be two options for Boolean values. Usually most people represent this as True or False, but it could also be 1 or 0, in accordance with binary.

Operations

Strings

Useful String Methods

```
name = "Python"

greeting = 'Hello'



# Concatenation

full = greeting + ", " + name #
Hello, Python
```

Print & User Input

```
s = " Hello, Python World! "



print(s.strip()) # Remove
whitespace

print(s.replace("Python",
"Beautiful"))

print(s.split(",")) # ['
```

```
# Length

print(len(name)) # 6


# Upper / Lower

print(name.upper()) # PYTHON

print(name.lower()) # python


# f-strings (modern way!)

age = 30

print(f"My age is {age}") # My
age is 30
```

```
Hello', ' Python...']

print(s.startswith(" Hello")) #
True

print(s.endswith("! ")) # True

print(s.find("Python")) # 9
(index)

print(s.count("l")) # 3

print("42".isnumeric()) # True

print("hello".capitalize()) #
Hello

print("hi".center(10, "*")) #
****hi****
```

Arithmetic

Standard Operators

```
a = 10

b = 3

print(a + b) # 13 — Addition

print(a - b) # 7 — Subtraction

print(a * b) # 30 —
Multiplication

print(a / b) # 3.333 — True
Division

print(a // b) # 3 — Floor
Division

print(a % b) # 1 — Modulus
(remainder)

print(a ** b) # 1000 —
```

Real examples

```
# Real example: km to miles

km = 100

miles = km * 0.621371

print(f"{km} km = {miles:.2f}
miles")
```

Exponentiation	

Operator	Symbol	Example
Addition	+	5 + 3 = 8
Subtraction	-	5 - 3 = 2
Multiply	*	5 * 3 = 15
Divide	/	5 / 2 = 2.5
Floor Div	//	5 // 2 = 2
Modulus	%	5 % 2 = 1
Power	**	2 ** 8 = 256

Assignment

Assignment Operators

```
Assignment operators are short cut operators to manipulate a
variable's values.
```

```
x = 10 # Basic assignment

x += 5 # x = x + 5 → 15

x -= 3 # x = x - 3 → 12

x *= 2 # x = x * 2 → 24

x /= 4 # x = x / 4 → 6.0

x //= 2 # x = x // 2 → 3.0

x **= 3 # x = x ** 3 → 27.0

x %= 5 # x = x % 5 → 2.0

# String multiplication!

line = "-" * 20

print(line) # --------------------

word = "Ha"

print(word * 3) # HaHaHa
```

Logic

Comparison

```
a, b = 10, 5

# Comparison (returns True or
False)

print(a == b) # False — equal

print(a != b) # True — not
equal

print(a > b) # True — greater
than

print(a < b) # False — less
than
```

Logical

```
a, b = 10, 5

# Logical operators

print(a > 5 and b < 10) # True
(both true)

print(a > 15 or b < 10) # True
(one is true)

print(not a > 5) # False
(flipped)
```

```
print(a >= 10) # True — greater or equal

print(a <= 9) # False — less or equal
```

Logic Tables

A	B	And	Or	Not (A)	XOR	(A OR B) AND NOT (A AND B)
0	0	0	0	1	0	0
0	1	0	1	1	1	1
1	0	0	1	0	1	1
1	1	1	1	0	0	0

Identity and Membership

Identity

```
# Identity operators: compare memory location

a = [1, 2, 3]
```

Membership

```
# Membership operators: check if in collection

fruits = ["apple", "banana",
```

```
b = [1, 2, 3]

c = a



print(a is c)     # True  —
same object!

print(a is b)     # False —
different objects

print(a is not b) # True
```

```
"mango"]



print("apple" in fruits)      #
True

print("grape" in fruits)      #
False

print("grape" not in fruits)  #
True



word = "Python"

print("Py" in word)           #
True
```